НЕ ФИНАЛЬНАЯ ВЕРСИЯ — WORK IN PROGRESS

Лекция 04.09.2026

База данных - организованная структурированная совокупность данных, хранящиеся в памяти ЭВМ и отображающих состояние объектов

Модель данных - набор концепций правил и структур, которые определяют как организуются хранятся и обрабатываются данные

СУБД - комплекс программных средств предназначенный для создания ведения и совместного использования БД.

Выбор модели данных критически важен – от него зависит производительность, гибкость

Лекция 11.09.2026

Реляционная БД:

Особенности реляционных БД:
- организует данные в виде двумерных таблиц
- Используют структурированный язык запросов SQL
- ACID - свойства

Про ACID:
1) Атомарность (Atomicity): Транзакция выполняется либо полностью, либо не выполняется вообще. Промежуточных результатов не бывает.
2) Согласованность (Consistency): База данных всегда переходит из одного корректного состояния в другое, не нарушая правила системы.
3) Изоляция (Isolation): Параллельные транзакции не влияют друг на друга; их результат выглядит так, будто они шли по очереди.
4) Устойчивость (Durability): Если транзакция успешно завершилась, изменения не пропадут, даже если случится сбой питания или системы

Преимущества:
- Благодаря механизму ключей, ограничений и транзакций, обеспечивается целостность и непротиворечивость данных.
- Использование SQL как стандарта облегчает разработку и поддержку приложений, а также миграцию данных между системами
- Процесс нормализации позволяет устранить избыточность данных и аномалии.

Недостатки:
- Изменение структуры таблиц на работающей системе может быть сложной и дорогой операцией.
- Горизонтальные масштабирование реляционных баз данных зачастую сложнее реализовать по сравнению с некоторыми NoSQL решениями.
- Сложные JOIN-запросы на огромных таблицах могут выполняться медленно.
- Неэффективна для хранения иерархических (древовидных) или графовых данных.

Два вида запросов:
- OLAP (Online Analytical Processing)
- OLTP (Online Transaction Processing)

Колоночная

Особенности:
- Вместо того чтобы хранить каждую запись целиком, данные организуются по столбцам.
- Реляционная таблица: строки и столбцы
- Фиксированная схема
- OLAP

Wide-column NoSQL
1) Масштабируемость записи
2) горизонтальная масштабируемость

Документоориентированная
- Данные хранятся в виде документов, а не в виде строк таблицы.
- Относятся к семейству NoSQL.
- Документы обычно представляют собой структуры в формате BSON
- Документы объединяются в коллекции (аналогичны таблицам, но без обязательного соответствия единой схеме для всех записей).
- Документы могут содержать вложенные объекты и массивы.
- Данные в виде агрегатов.
- Горизонтально масштабируема.

Графовая
- Вместо таблиц - ноды, связи, свойства. Каждый узел - json.
- Скейлинг по горизонтали.
- Нет жестко фиксированной схемы.
- Узкоспециализированная БД, недостатков нет
- Соц сети, рекомендации, цепочки поставок, базы знаний.

K-V БД
- Похожи на документоориентированные.
- Данные организованы в виде пар ключ-значение.
- Значение может быть любым.
- Высокая производительность, низкая надежность + низкая агрегация.

Timeseries (Временные ряды)
- Данные упорядоченные по времени.
- Высокая скорость записи + агрегации и интерполяции.
- Узкоспециализированная.

Векторная
- Данные хранятся в виде векторов вещественных чисел (ML-задачи).
- Оптимизация через спец индексы и прочие ML-штуки.

Хранилище данных

- предназначены для работы с файлами, такими как документы, изображения, видео, аудио файлы
- располагаются в каталогах и подкаталогах.
- Хранит помимо файла – метаданные
- Доступ к файлам осуществляется по их именам и путям в файловой системе
- Необходимо так как в обычных БД размер кортежа ограничен. Поэтому в БД не хранят файлы

Лекция 18.09.2026

Погружение в реляционные СУБД

- Отношение (табличка) – фиксированный набор (множество) атрибутов, каждый из которых определен на некотором домене. Изменяемое множество кортежей, соответствующих заголовку. Каждый кортеж содержит множество значений атрибутом.
  - Отношение не упорядочено
  - Атрибуты не упорядочены (?)
  - Все кортежи уникальны (В SQL не обязательно)
- Атрибут (строчка) – именованная характеристика или свойство некоторого объекта, информация о котором хранится в отношении. Каждый атрибут определяется на некотором домене. Домен - это множество всех допустимых значений атрибута.
  - У каждого атрибута свое уникальное имя внутри таблицы (отношения).

  Виды атрибутов:
  - Простой
  - Составной
  - Однозначный
  - Многозначный
  - Производный

  В SQL допустимы только простые однозначные атрибуты. Иначе нарушаются нормальные формы, что приводит к противоречивости данных. Тогда если атрибуты в таблице не однозначные простые — ошибка в проектировании.

  > **Важно понимать про атрибуты**
  >
  > - В реляционной модели допустимы только простые однозначные атрибуты
  > - Составные атрибуты при переходе к реляционной модели разбиваются на отдельные столбцы или выносятся в отдельную таблицу
  > - Многозначные атрибуты выносятся в отдельную таблицу и связываются с основной через внешний ключ
  > - Производные атрибуты обычно не хранятся в базе, а вычисляются в запросах или представлениях. (могут храниться для оптимизации)

- Кортеж (колонка)

Definition

Домен - это множество всех допустимых значений атрибута.

- Тип Данных + ограничение
- Целостность данных.
- Согласованность

Целостность - набор правил, которым должны удовлетворять данные, чтобы соответствовать предметной области. Это требования к данным. Согласованность - состояние БД, при котором все ограничения целостности выполнены. То есть данные не противоречивы.

Целостность бывает:

- Доменной (смысловая)
- Целостность сущностей
- Ссылочная
- Пользовательская

Важные определения

- Степень отношения (арность) – количество атрибутов в отношении
- Кардинальность отношения — количество кортежей в отношении
- Кардинальность атрибута — количество уникальных значений атрибута
- Селективность — принимает от 0 до 1 , кардинальность атрибута / кардинальность отношения. Если селективность близка к 1 – большая часть значений уникальная, хорошая оптимизация через индексацию. И наоборот, если близка к 0 — атрибуты это switch-и (например мужской/женский пол). Нет смысла оптимизировать.

Еще важные определения

- Суперключ – некий набор атрибутов, по которому можно уникально идентифицировать запись. Их может быть много
- Потенциальный ключ – подмножество суперключа, минимальный набор атрибутов, который нужен для уникальной идентификации кортежа.
- Первичный ключ – СУБД проверяет что не NULL и что уникален, желательно чтобы не менять. На этот ключ накладывается constrains.
- Внешний ключ — ссылка на первичный ключ.
- Альтернативный ключ - потенциальный ключ, который не был выбран в качестве Первичного.

Типы Primary Key

- Естественный ключ – хранит характеристику объекта
- Суррогатный ключ – значение не связанное с объектом, поэтому не измется – ведь оно ничего не значит. Лучше использовать их (неизменяемость). Бывают:
  - Целочисленный
  - Uuid. Хорошая идея, когда:
    - Микросервисы
    - Генерация на front.
- Простой ключ – состоит из одного атрибута
- Составной ключ - взяты несколько атрибутов, из них сделан PK. Плохо, так как внешние ключи в других таблицах будет иметь также несколько атрибутов, накладные расходы увеличены.

Типы данных (в SQL)

- Целочисленный (smallint, int, bigint)
- Floating point (real, double)
- Строки (varchar, char, text)
- Дата и время (date, time, timestamp, interval)
- Логические (bool)
- Геометрические типы (point,path,line)
- json

Целочисленные типы (в SQL)

- Фиксированные (smallint, int, bigint)
- DECIMAL(M,D)
- Floating Point
- Денежный
- Автоинкрементирующиеся

Биты (в SQL)

- BIT(n)
- VARBIT - переменной длины
- BOOLEAN - TRUE,FALSE,NULL.

Строки (в SQL)

- char(n) – строка фиксированной длины
- varchar – переменной длины
- text – строковые данные без ограничения длины
- bytea – двоичные данные без ограничения длинны

Дата и время (в SQL)

- TIMESTAMP [без часового пояса]
- TIMESTAMPZ [с часовым поясом]
- DATE
- TIME [без часового пояса]
- TIME [с часовым поясом]
- INTERVAL

- DDL- Data definition language
- DML - Data manipulation language

Основные команды DDL

- CREATE TABLE
- ALTER TABLE
- DROP
- TRUNCATE

Основные команды DML

- SELECT
- INSERT
- UPDATE
- DELETE
- MERGE

Логический порядок выполнения:

1) WITH
2) FROM
3) WHERE
4) GROUP BY
5) HAVING
6) WINDOW
7) SELECT (вычисление выражений | DISTINCT)
8) UNION/INTER/EXCEPT
9) ORDER BY
10) LIMIT/OFFSET/FETCH
11) FOR UPDATE

WITH - именованный подзапрос, рекурсивные подзапросы / оптимизация. А-ля функция.

При удалении больших объемов данных, TRUNCATE лучше DELETE (меньше записей в журнале записей, не проверяет данные, сразу удаляя их).

Лекция 25.09.2026

Реляционная алгебра

Типы операций:

- Теоретико-множественные (объединение, разность, пересечение, дек. произведение)
- Специальные реляционные (выборка σ, проекция π, соединение , деление $\%$)

Лекция 02.10.2026

Нормализация

Дублирование $\neq$ избыточность. Избыточность — вероятность получить аномалии. Дублирование может быть нормальным, а избыточность — нет. Избыточность — данные хранятся несколько раз, но при этом эти данные можно получить из другого места. Пример: возраст, его можно получить из даты рождения. Пример дублирования: user_id.Их несколько, но нельзя получить из другого места.

Тут слайды из презы про степени нормализации

Лекция 09.10.2026

Транзакции

Транзакция – некоторая последовательность операций, которая выполняется как единое целое. Транзакция либо завершается успешно (COMMIT), либо полностью отменяется (ROLLBACK), чтобы гарантировать целостность данных. Свойства транзакции (ACID):

1) Атомарность (Atomicity): Транзакция выполняется либо полностью, либо не выполняется вообще. Промежуточных результатов не бывает.
2) Согласованность (Consistency): База данных всегда переходит из одного корректного состояния в другое, не нарушая правила системы.
3) Изоляция (Isolation): Параллельные транзакции не влияют друг на друга; их результат выглядит так, будто они шли по очереди.
4) Устойчивость (Durability): Если транзакция успешно завершилась, изменения не пропадут, даже если случится сбой питания или системы

Ограничения целостности:

- NUT NULL
- UNIQUE
- Primary Key
- Foreign key
- CHECK
- EXCLUSION CONSTRAINS

Isolation

Изоляция транзакций:

- READ UNCOMMITED - Позволяет читать даже не завершенные изменения других транзакций (не используется в Posgre)

- READ COMMITED (по умолчанию) — Транзакция видит только зафиксированные (COMMIT) изменения других транзакций.
- REPEATABLE READ — Транзакции вдят данные в том виде, в каком они были на моменте её начала, даже если другие транзакции вносят изменения
- SERIALIZABLE — Максимальный уровень изоляции. Гарантирует

Виды аномалий:
- Потерянное обновление
- Dirty Read
- Nonrepeatable Read
- Phantom Read
- Serialization Anomaly
- Циклическая зависимость данных (Write Skew)

Проверка на аномалии – NP полная задача, поэтому полную блокировку (Предикатную) не используют. На практике используется:
- Блокировка диапазонов ключей: Наиболее распространенная практика. При сканировании диапазона транзакция блокируется не только существующие записи, но и «промежутки» между ними, а также запись, следующую за диапазоном. Это предотвращает вставку новых записей внутрь заблокированного диапазона.
- Блокировка индексов или страниц: Более грубый подход – блокировка целых страниц индекса или даже таблиц, что гарантирует отсутствие фантомов, но ценой снижения производительности.

PostgreSQK испрользует оптимистичный подход, основанный на Serializable Snapshot Isolation.

Durability

Write-Ahead Logging

стратегия, при которой все изменения данных сначала записываются в журнал, и только потом применяются к основным файлам баз данных.

Структура WAL:
- Header (24 байта)
- Page ID (8 байт)
- Old Data (размер страницы)
- New Data (размер страницы)
- Checksum (8 байт)

Работает не всегда, и зависит от конфигурации PostgreSQL:
- fsync = on — при отключении WAL файл хранится в оперативной памяти.
- synchronous_commit = on — при отключении, данные будут в асинхронном режиме (подтверждение коммита будет отправлено до записи на диск)
- wal_buffers = …MB

Checkpoint — процесс при котором все измененные данные из буферного кэша сбрасываются на диск, создавая согласованное состояние БД.

Multi-Version Concurrency Control (MVCC)

Механизм для изоляции транзакций, но с минимальным количеством блокировок (для параллельной работы) [возможно нуждается в правках] Условно: В каждой таблице создается несколько доп атрибутов, важные для MVCC: $x_{\min}$— номер транзакции при котором запись была внесена и $x_{\max}$. — номер транзакции при которой запись была удалена. Тогда для проверки существует ли запись, транзакции нужно проверить что $x_{\min} >$ текущего номера, а $x_{\max} <$ текущего номера.? Тогда SELECT просто проверяет это неравенство, INSERT создаст новую запись куда присвоит свой номер атрибуту $x_{\min},$) DELETE поставит свой номер в значение $x_{\max}$, а UPDATE установит в $x_{\max}$ старой записи свой номер, и создаст новую запись с $x_{\min} =$ текущий номер.

Лекция 16.10.2026

Индекс - объекты базы данных, предназначенные для ускорения поиска и выборки строк из таблицы, минимизируя количество данных, которые нужно просматривать. Также индексы служат для поддержания некоторых ограничений целостности.

Как устроены? Устанавливает соответствие между ключем и строками таблицы, в которых этот ключ встречается. Строки идентифицируются с помощью TID (tuple ID).

Однако проблема - увеличение затрат на запись.

Типы индексов;
- Btree
- Hash
- GIN
- GiSt
- SP-Gist
- BRIN

[Здесь должно быть объяснение всех индексов до BRIN]

Лекция 23.10.2026

LSM-Tree

[Здесь должно быть объяснение LSM-Tree] Архитектура PGS Ролевая модель

Лекция 30.10.2026

Этапы выполнения запроса. Лексический разбор.

На этом этапе сервер принимает SQL-текст запроса и анализирует его на наличие синтаксических ошибок. Результатом является дерево разбора (parse tree) представляющее структуру запроса.

Семантический разбор

- Заменяет представления на их определения или обрабатывает правила, определенные для таблиц.

- Проверка прав у пользователя к этим объектам.
- Вся необходимая для семантического анализа информация хранится в системном каталоге.
- Семантический анализатор получает от синтаксического анализатора дерево разбора и перестраивает его, дополняя ссылками на конкретные объекты базы данных, указанием типов данных и другой информацией.

Фактически тот же самый parse tree, но уже с физическими представлениями и проверенными правами доступа.

Планирование.

- Любой запрос можно выполнить разными способами.
- План выполнения представляется в виде дерева, но его узлы содержат не логические, а физические операции над данными.
- Текстовое представление плана выводит команда EXPLAIN (ANALYSE). Нельзя делать EXPLAIN ANALYSE на delete, потому что он одновременно показывает план и выполняет запрос.
- Скорость выполнения будет зависеть от порядка JOIN.
- В Postgre есть ограничение — если у вас больше [20] запросов, планировщик перестанет пытаться подбирать оптимальный порядок, а возьмет рандомный.

Планирование 2.

- Планировщик принимает переписанное дерево запроса и генерирует один или несколько вариантов его выполнения.
- Оптимизатор оценивает стоимость каждого варианта плана, используя статистические данные о таблицах, и подбирает наиболее выгодный план.

Планы

- Количество возможных планов экспоненциально зависит от количества соединяемых таблиц.
- Для сокращения пространства перебора традиционно используется алгоритм дин проги в сочетании с некоторыми эвристиками. (генетический алгоритм?)
- Точное решение задачи оптимизации не гарантирует что найденный план действительно будет лучшим.

Типы планов

- Общий план. Если несколько раз использовать план с одними константами, планировщик перейдет к общему плану, который вместо констант заменит их на переменные.
- Частный план. План строится под конкретный запрос с конкретными константами.

Общие и частные планы

- При первом выполнении строится частный план с учетом параметров.
- При неоднократном выполнении планировщик перейдет к общему плану, который за счет кэширования снизит затраты на его выполнения.

Выполнение

- После выбора оптимального плана исполнитель запускает последовательность операций, предусмотренных планом. Он последовательно проходит по узлам плана
- При выполнении запросов осуществляется доступ к данным через буферный менеджер, который минимизирует количество операций ввода-вывода на диск.
- Если запрос рассчитан на параллельное выполнение результат собирается из нескольких рабочих процессов и объединяется в итоговый набор данных.

Статистика. Определение

Это набор данных которые собираются дсистемой для оценки распределения значений в таблицах и столбцах

хранения

- Хранится в системной таблице pg_statistic
- Прямой доступ обычно не требуется, и для просмотра статистики чаще используется представление pg_stats.
- Это публичное представление которое предоставляет информацию из pg_statistics в удобном для чтения виде, без избыточных деталей, доступных только суперпользователям.
- Базовая статистика уровня отношений хранится в таблице pg_class.

Данные

- reltuples: число кортежей в отношении
- n_distinct: оценка количества уникальных значений (*Польза:* если у нас малое количество уникальных значений, нет смысла строить индекс).
- most_common_vals и most_common_freqs: список наиболее часто встречающихся значений и их частоты
- histogram_bounds: границы гистограммы распределения значений, которые используются для оценки селективности диапазонных запросов.
- correlation: оценка корреляции между порядком значений в столбце и порядком их хранения. Оно же хаотичность данных. Если все положительные
- null_frac: Доля неопределенных значений.

Сбор статистики

- Статистика собирается при анализе, ручном или автоматическом. Однако ввиду особой важности базовая статистика рассчитывается также при выполнении некоторых операций (VACUUM FULL, CLUSTER, CREATE INDEX, REINDEX) и уточняется при очистке
- Для анализа случайно выбираются 300 x default_statistics_target строк. Поскольку размер выборки достаточной для построения статистики заданной точности, слабо зависит от объема анализируемых данных, размер таблицы не учитывается. Фактически если у нас количество данных >30k, мы будем рассматривать только 30k. Но из-за теоремы? эту выборку можно считать такой же точной как если бы мы шли по всей выборке.

pg_stats

- Для уточнения оценки при неравномерном распределении собирается статистика по наиболее часто встречающимся значениям и частоте их появления. Представление pg_stats показывает два этих массива в столбцах most_common_vals и most_common_freqs.

- Список частых значений используется и для оценки селективности условий с неравенствами. Например для условий вида «столбец < значение» надо найти most_common_vals все значения, меньше искомого, и просуммировать частоты из most_common_freqs.

На примере как рассчитать статистику. Пусть нужно найти where age > 20. Тогда используя pg_stats, не выполняя вычисления в запросе, можно: retuples $*\sum$ most_common_vals[20...max]. Минус – на больших размерах промежутков, вычисление невозможно. Решение - most_common_freqs.Тогда мы храним не точные значения, а диапазоны,и рассчитываем вероятность.

Для работы с не границами диапазонов, а элементами внутри, гарантируется что внутри бакетов работает *равномерное распределение.*

pg_stats 2

- Когда число различных значений слишком велико чтобы записать их в массив, на помощь приходит гистограмма

pg_stats 3

- Поле corellation представления pg_stats показывает корелляцию между физическим расположением данных и логическим порядком в смысле операций сравнения.
- Чем хаотичнее расположены данные физически, тем менее эффективно будет работать индекс.

Табличные методы доступа. Последовательное сканирование.

- Полностью читается файл основного слоя таблицы. На каждой прочитанной странице проверяется видимость каждой версии строки.
- Чтение происходит через буферный кеш; чтобы большие таблицы не вытесняли полезные данные. При этом другие процессы, одновременно сканирующие ту же таблицу, присоединяются к кольцу и тем самым экономят операции дисковых чтений. Поэтом в общем случае сканирование может стартовать не с начала файла.
- Последовательное сканирование — самый эффективный способ прочитать всю таблицу или значительную ее часть. Иными словами, последовательное сканирование работает хорошо при низкой селективности.

Последовательное сканирование, part 2.

- В оценке стоимости оптимизатор учитывает дисковый ввод-вывод и ресурсы процессора.
- Стоимость ввода-вывода рассчитывает как произведение числа страниц в таблице на стоимость чтения одной страницы, при условии что страницы читаются последовательно.
- Соотношение по умолчанию подходит для HDD-дисков; для накопителей SSD имеет смысл существенно уменьшить значение параметра random_page_cost (значение seq_page_cost, как правило, не трогают, оставляя единицу в качестве опорного значения).
- Оценка ресурсов процессора учитывает стоимость обработки каждой версии строки (которая определяется для планировщика значением параметра cpu_tuple_cost)
- Если на сканируемую таблицу наложены условия, они отображаются в плане запроса под узлом Seq Scan в секции Filter. Оценка числа строк будет учитывать селективность этих условий, а оценка стоимости — затраты на их вычисления.

Parallel seq scan

- Чтение выполнение несколькими параллельно работающими процессами. Процессы синхронизируются между собой с помощью специально отведенного участка общей памяти, чтобы не прочитать одну и ту же страницу дважды.

Gather

- собирает результаты параллельных рабочих процессов
- Gather Merge: В этом случае параллельно рабочие процессы выполняют сортировку локально, а Merge объединяет отсортированные потоки, гарантируя, что итоговый результат также будет отсортированный без выполнения доп отсортировки на этапе объединения.

Сканирование только индекса

- Операция представляется в плане запроса узлом Index Only Scan.
- На оценку сканирования только индекса влияет доля табличных страниц, отмеченных в карте видимости.
- Если мы после index scan должны далее обращаться к памяти бд для чтения того, что не лежит в индексе. Тогда это называется просто Index Scan.

Сканирование по битовой карте

- Ограничение индексного сканирования связано с тем, что при уменьшении корреляции увеличивается количество обращений к страницам, а характер чтения меняется с последовательного на случайный.

Сравнение методов доступа

- Зависимость стоимости различных методов доступа от селективности условий можно представить следующим образом.

Стоимость индексного сканирования сильно зависит корреляции. При идеальной корреляции индексное сканирование эффективно даже при довольно большой доле выбираемых строк.

Виды и способы соединений . Соединение вложенным циклом.

- На эффектность соединения вложенным циклом влияет несколько условий: кардинальность внешнего набора строк, наличие метода доступа ко внутреннему набору, позволяющего эффективно получить нужные строки, повторные обращения к одним и тем же строкам внутреннего набора.
- В общем случае полная стоимость соединения складывается: из стоимости получения всех строк внешнего набора, однократной стоимости первоначального получения всех строк внутреннего набора (в ходе которого выполняется материализация), (N-1) кратной стоимости

Виды реализации Join:

- Nested loop $(O(n \cdot m))$

```
for r in R:
  for s in S:
    if P(R.r, S.s):
      print(r+s)
```

- HashTable:

```
let R = min(R,S)

for each row r in R:
hashTable[hash(r.a)].append(r)

for each row s in S:
    bucket = hashTable[hash(s.b)]
    for each row r in bucket:
        if r.a == s.b:
            print(r + s)
```

Асимптотика: $O(n)$.

Строим хэш таблицу по меньшей таблице. Проходимся по второй, если не равны то добавляем в result (если нет коллизии).

- Merge:

```
R.sort(a)
R.sort(b)
while not endof(R) and not endof(S)
  if R.a < S.b:
    next(R)
  if R.a > S.b:
    next(S)
  if R.a == S.b
    print(r+s)
    next(S)
```

Асимптотика: $O(N \log N + M \log M + (N + M))$

Дорого, но если не нужно сортировать (проверка по индексу), то очень даже ничего. Соединение слиянием работает для наборов данных, отсортированных по ключу соединения, и возвращает отсортированный результат. Входной набор может оказаться уже отсортированным в результате индексного сканирования, или он может быть отсортирован явно.

Используется быстрая сортировка, частичная пирамидальная (когда почти отсротированный массив), внешняя сортировка слиянием (для больших chunks).

Наиболее эффективный способ — хэш таблица, так как можно распаралеллить.

PostgreSQL использует quicksort, чтобы выполнить операцию в оперативной памяти. Размер выделенной памяти для сортировки контролируется паараметром work_mem.

Агрегация

Агрегация тоже может выполняться:

- HashAgregation. Для каждой входной строки СУБД вычисляет хеш от ключей группировки (GROUP BY) и строит хеш таблицу. Если группа уже есть в хеш-таблице, её значения (сумма, количество, минимум/максимум) обновляются. Если нет — создаётся новая запись.В отличие от альтернативного подхода данные не нужно предварительно сортировать, что экономит время при больших объемах не упорядоченных данных.

- GroupAgregation. Сначала

Explain

Фактически EXPLAIN состоит из:

- Тип операций (узлов) плана
- Оценки стоимости
- Оценка количества строк
- Размер данных
- Параллельное выполнение
- Фактическое время выполнения. Время, затраченное на выполнение каждой операции. (Только если EXPLAIN ANALYSE)
- Фактическое количество строк. (Только если EXPLAIN ANALYSE)
- Число циклов/итераций.

Лирическое отступление про индексы в других СУБД

Иногда, в других СУБД, индекс определяется как первичный, вторичный, частичный, и кластерный. Концептуально кластерный индекс это индекс в котором сортируются данные. Он невозможен в PostgreSql (потому что разные версии данных существуют). Первичный вторичный также концептуально ничем не отличаются, по дефолту на Primary key создается индекс для поддержания уникальности. Частичный (разряженный) индекс — индекс созданный по горячим (использующимся) данным.

Lecture 6.11.2026

Безопасность. Две притчи из актуальных на момент написания событий про безопасность:

> *“ Люди делятся на 4 типа: те кто не делают бекапы, те кто уже делают бекапы, те кто уже делают бекапы и валидируют их, и те кто уже делают бекапы, валидируют их и хранят в разных местах ”*
>
> ***— Мацнев Н.И.***

> *“ Изначально бекапы хранились на YandexCloud... Потом на Beget.... ”*
>
> ***— Мацнев Н.И.***

Что такое безопасная система?

Безопасная система обеспечивает конфиденциальность, целостность и доступность данных и функций, защищая их от несанкционированного доступа, раскрытия, и т.д.

CIA Triad

- Конфиденциальность. Данные доступны только тем, кому разрешено. Шифрование данных, строгий контроль доступа.
- Целостность. Защита от несанкционированных изменений. Использование цифровых подписей, хеширование, контроль версий, а также поддержание цепочки поставок и журналирование всех изменений.
- Доступность. Обеспечение бесперебойного доступа. Резервное копирование и восстановление данных, балансировка нагрузки, отказоустойчивость системы, защита от вредоносного ПО.

Определение пользователя

- Идентификация — процесс, когда информационная система определяет, сущетсвует конкретный пользователь или нет. Делает она это с помощью идентификатлора. Идентификатором может быть логин, электронная почта, номер телефона или другой признак котоырй есть только у одного пользователя
- Аутентификация — процесс, когда пользователь вводит ключ, например пароль или пин код подтверждая свое право на доступ к той или иной учетной записи и хранящейся в ней информации.
- Авторизация — процесс предоставления пользователю или системе определенных прав и разрешений для доступа к ресурсам (файлам, функциям, настройкам) после того, как он успешно прошел проверку

AAA (Authentification, Authorization, Accounting)

- Accounting — сбор и хранение свидетельств действий субъектов в системе.
- Фиксируем кто, что и когда сделал
- Нужно для расследования инцидентов

Аутентификация (Виды)

- То что субъект знает (по факту пароль)
- То что субъекту принадлежит (одноразовый код)
- То что является неотъемлемой характеристикой субъекта (отпечаток пальца | иная биометрия)

Многофакторная аутентификация

- Если для входа требуются пароль и, например, ответ на секретный вопрос (который тоже является знанием), то это не многофакторная аутентификация, а всего лишь двухэтапная проверка в рамках одного фактора.

Авторизация

- можно сделать матрицей USER - OBJECT. Но это взрыв по памяти.
- Другой подход это хранить списками, тогда памяти тратится меньше, но поиск за линию.

ПОлитики авторизации

- Кто управляет правилами?
  - DAC Реализуется через ACL и владение.
  - MAC Метки безопасности — это атрибуты, которыми управляет система и которые пользователь не может менять.

- Каким образом организуем контроль?
  - RBAC (Role-Based Access Control)
  - ABAC (Attribute-Based Access Control)
  - ReBAC (Relationship-Based Access Control)

Дискреционное управление доступом

- Каждый объект имеет владельца.
- Владелец полностью контролирует доступ к своему объекту.
- Правда доступа определяются на основе списков контроля доступа (ACL)
- Существует протекание доступов – если у меня есть доступ на чтение, я могу дать его другому, и так до бесконечности. Это плохо. Поэтом доступы делают на два типа:
  - Доступ к самому объекту.
  - Доступ к выдаче доступов.

Уровни привилегий:

- SYSTEM PRIVILEGES
- DATABASE PRIVILEGES
- SCHEMA PRIVILEGES
- TABLE PRIVILEGES
- COLUMN PRIVILEGES
- ROW PRIVILEGES

Модель доступа в SQL

- Создание CREATE ROLE {Name} {attr1, attr2, ...}
- Изменение ALTER ROLE developer WITH PASSWORD
- USER
- SUPERUSER
- DATABASE CREATOR

[Много всякого про ролевой доступ в SQL]

Модель дсотупа. Привиллегии таблиц

- SELECT
- INSERT
- UPDATE
- REFERENCES
- DELETE
- TRUNCATE
- TRIGGER. Функция которая выполняется на какое-то действие в определенном отношении.

* Update не сработает без SELECT.

Привилегии БД

- CREATE (Создание схем)
- CONNECT (Чтение)
- TEMPORARY (Создание временных таблиц, не логируются)

Стандартные БД

- Когда создается новая БД без явного указания шаблона, используется БД template1.
- Template0 - это «чистая» БД, которая не изменялась после установки PostgreSQL. Она содержит минимальное количество объектов и служит эталоном.

Привилегии схем

- CREATE
- USAGE
- CREATE на схему разреает создание новых объектов. USAGE разрешает обращаться к существующим объектам, но не дает прав на операции над ними. Таким образом USAGE — является необходимым, но не достаточным уровнем.

Схема public

- Создается по умолчанию при создании новой БД
- Объекты в ней доступны всем пользователям, если не настроены ограничения
- При создании объектов не указана какая-либо другая схема, они автоматически создаются в public.

Мандатный управление доступом

- Каждому субъекту и объекту присваивается уровень допуска (например: открыто, секретно, совершенно секретно) и опционально категории (для горизонтального разделения по отделам или проектам)
- Каждому пользователю также выданы уровни доступа.
- Правила чтения и записи:
  - Чтение (No Read Up): субъект может читать данные объекта, только если его уровень допуска не ниже уровня конфиденциальности объекта («нет чтения вверх»).
  - Запись (No Write Down): субъект может записывать данные в объект, только если его уровень допуска не выше уровня объекта («нет записи вниз»), чтобы предотвратить утечку информации
- «Нет права давать права»

Шифрование БД

- Прозрачное шифрование.
- На уровне столбцов.
- На уровне приложений.
- По умолчанию сквозное шифрование кроме sqlite.

- Наименьшие привилегии: каждый субъект получает только необходимый минимум прав и на необходимое время, по умолчание — запрещено
- Разделение обязанностей: критичная операция распределена между несколькими ролям.